



IntegrationOps AI

Autonomous SAP CPI Incident Intelligence Platform

Hackathon build · Enterprise reliability · AI-first operations

- Rashmi Joshi



Problem

Integration failures drain time and trust

SAP CPI Message Processing Logs are dense — RCA is expert-led and slow.

Manual correlation across MPL, design-time artifacts, and tickets is error-prone.

Mean time to resolve stays high when context is fragmented across tools.

◇ Solution

One autonomous agent, one structured outcome

Autonomous investigation agent — same pipeline as operator “deep dive”.

Pulls CPI runtime logs + iFlow metadata (OData) into a single LLM briefing.

Returns RCA, severity, confidence, and recommendations as enterprise JSON.

Architecture

End-to-end signal → decision → action surface



OData MPL + design-time metadata → FastAPI → run_investigation → build_context → OpenRouter (DeepSeek / fallback) → structured RCA
→ SQLite + mock inbox + Jira-ready fields

✦ Key Features

Shipped in the MVP + clear hooks for scale

- © Real-time / scheduled CPI FAILED-MPL monitor (APScheduler) with dedupe.
- © AI-driven RCA + confidence scoring (OpenRouter + heuristic fallback).
- © Persisted incidents (SQLite) + observability lifecycle (LLM audit join).
- © Jira-ready fields + mock alert inbox (email preview without SMTP).
- © Operator dashboard + “run monitor now” for instant demos.

Live Demo Flow

What judges see in under two minutes

- ① Incident detected — FAILED MPL in lookback window.
- ② Logs + metadata fetched — CPI OData into the agent.
- ③ AI analysis — LLM JSON with evidence-aligned confidence.
- ④ RCA generated — dashboard + mock inbox + SQLite audit trail.
- ⑤ Jira + email — schema-ready / mock UI today; wire to ITSM tomorrow.

⚡ Tech Stack

Modern, boring-in-a-good-way production defaults

FastAPI · Uvicorn · Pydantic — typed APIs, OpenAPI docs, /api prefix for proxies.

React 18 + Vite + React Router — dashboard, lifecycle, mock inbox.

OpenRouter (DeepSeek + Llama fallback) — httpx, JSON-mode prompts.

SAP CPI OData (MPL + Integration Content) — mock + live paths.

Render (API) + Vercel (UI) — env-driven VITE_API_URL, CORS-ready.

↑ Impact

Why operators and leadership care

- ↓ MTTR — first-pass triage with cited evidence, not tribal knowledge.
- ↑ Consistency — same agent steps every incident; auditable LLM exchanges.
- ↑ Reliability posture — confidence scores + severity for prioritization.

→ Future Scope

From insight to autonomous remediation

Auto-healing playbooks — safe rollbacks, cache clears, credential rotation hooks.

Event-driven remediation — CPI + ITSM webhooks feeding a closed loop.

Multi-system observability — extend the agent beyond CPI (Mulesoft, Workato, Workday).



IntegrationOps AI

Autonomous SAP CPI Incident Intelligence

[Live API](#) · [Dashboard](#) · [Mock inbox](#) · [Observability drill-down](#)

Demo Screenshots

Autonomous SAP CPI Incident Intelligence

The screenshot displays the SAP CPI interface for a test execution. The top section shows the request configuration: a POST method to the URL `https://443/http/Pattern/ContentBasedRouting/RaiseError`. The request body is in raw XML format, containing a `<PurchaseOrder>` element with shipping and billing addresses. The bottom section shows the response, which is a `500 Internal Server Error`. The response body contains the message: "An internal server error occurred: The MPL ID for the failed message is : AGn_ywqYTfRdG7SQUxGFxo1ZB0fq".

Source Application Triggers Interface

Received Interface failure in response

Annotations in the image:

- 1: POST method
- 2: URL
- 3: Body tab
- 4: Send button
- 5: Body tab in response section
- 6: 500 Internal Server Error status
- 7: Response body text

Demo Screenshots

Autonomous SAP CPI Incident Intelligence

SAP

Integration Suite

27

Overview / Monitor Message Processing

Message Status Overview

Hide Filter Bar

Time: Past Hour

Status: All

Type: All

Package: All

Artifacts: All

or

ID: Message, Correlation or Application Message

May 09, 2026, 16:05:53 - May 09, 2026, 17:05:53

Messages (2)

<< < 1 / 1 > >> ↺

Artifact Name	Status
Pattern Content Based Routing - Raise Error	Failed
May 09, 2026, 17:02:18	31 ms
Pattern Content Based Routing - Raise Error	Failed
May 09, 2026, 16:31:03	68 ms

Pattern Content Based Routing - Raise Error

Last Updated at: May 09, 2026, 17:02:18

Status

Error Details

Properties

Logs

Artifact Details

Status

Message processing failed.

Processing Time: 31 ms

Error Details

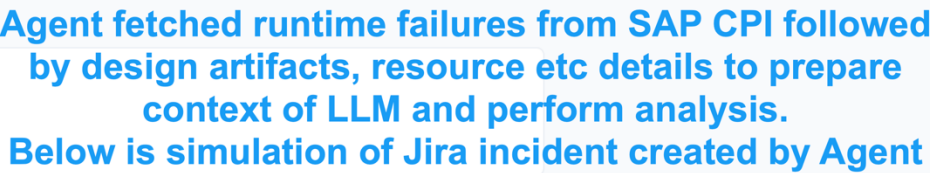
Last Error

com.sap.it.rt.adapter.http.api.exception.HttpResponseException: An internal server error occurred: No consumers available on endpoint: /Pattern/GenericReceiverDataStore. Exchange[]. The MPL ID for the failed message is : AGn_ywqYTfRdG75QUxGFxo1ZB0fq

Error occurred on SAP
CPI Platform

Autonomous SAP CPI Incident Intelligence

Autonomous SAP CPI Incident Intelligence



Demo Screenshots

Autonomous SAP CPI Incident Intelligence

IntegrationOps-AI

SAP CPI monitoring & incident analysis

Dashboard

Mock alert inbox

Monitor lifecycle

Mock alert inbox

Preview-only email UI — no SMTP or mail server. Messages are built from [Dashboard](#) incidents when available; otherwise sample CPI alerts are shown. Recipient address comes from GET /health (mock_alert_email / MOCK_ALERT_EMAIL in backend/.env).

Refresh

INBOX · 2 · FROM INCIDENTS SQLITE

From monitor

[MEDIUM] CPI incident — Pattern_ContentBasedRouting_RaiseError

Insufficient detail in runtime logs to determine specific failure cause; Status=FAILED is too generi...

May 9, 2026, 5:03 PM

From monitor

[MEDIUM] CPI incident — Pattern_ContentBasedRouting_RaiseError

The logs indicate a generic failure with status=FAILED, but no specific error details or adapter inf...

May 9, 2026, 4:31 PM

Mock client · Not sent

From IntegrationOps-AI Alerts <alerts-noreply@integrationops-demo.local>

To integrationops-alerts@demo.example.com

Date May 9, 2026, 5:03 PM

[MEDIUM] CPI incident — Pattern_ContentBasedRouting_RaiseError

Autonomous monitor stored this investigation in SQLite.

Integration artifact (iFlow)	Pattern_ContentBasedRouting_RaiseError
Message ID (MPL)	AGn_ywqYTfRdG7SQUxGFxo1ZB0fq
Severity	MEDIUM
Error type	UNKNOWN
Confidence	45%
Status	completed
Jira	—

ROOT CAUSE

Insufficient detail in runtime logs to determine specific failure cause; Status=FAILED is too generic

RECOMMENDATION

Enable detailed logging for the iFlow and reproduce the issue to capture more specific error details

In production you would wire this HTML to SendGrid, Amazon SES, or Microsoft Graph — this page is a static preview only.

Agent simulated an alert email communication for the business to get notified

Demo Screenshots

Autonomous SAP CPI Incident Intelligence

Monitor & agent lifecycle

Each row is one persisted incident (MPL MessageGuid). The pipeline shows how the backend agent processed that failure; llm_exchange rows are the exact audit trail from llm_audit.sqlite. Matching rows from the in-process monitor buffer appear under monitor runs (cleared on uvicorn restart).

Refresh

 Incidents DB: on · LLM audit: on

Incidents (2)

- AGn_ywqYTRdG7SQUxGFxo1ZBOfq**
Pattern_ContentBasedRouting_RaiseError · MEDIUM · UNKNOWN · 1 LLM row(s)
- AGn_w7fqJbs2TS23Gh17NQGwc9Bx**
Pattern_ContentBasedRouting_RaiseError · MEDIUM · UNKNOWN · 1 LLM row(s)

Detail

SQLite incident row

```
{
  "id": 2,
  "timestamp": "2026-05-10T00:03:58.528723+00:00",
  "message_id": "AGn_ywqYTRdG7SQUxGFxo1ZBOfq",
  "iflow": "Pattern_ContentBasedRouting_RaiseError",
  "error_type": "UNKNOWN",
  "severity": "MEDIUM",
  "root_cause": "Insufficient detail in runtime logs to determine specific failure",
  "recommendation": "Enable detailed logging for the iflow and reproduce the issue",
  "confidence_score": 45,
  "jira_ticket_id": null,
  "investigation_status": "completed",
  "llm_exchange_count": 1
}
```

Agent pipeline (canonical)

These four steps always run inside run_investigation before anything is written to llm_exchange.

- Fetch runtime logs** (fetch_logs)
CPI Message Processing Logs (MPL) for the integration artifact and MessageGuid.
- Fetch iflow metadata** (fetch_metadata)
Design-time OData: artifact, configurations, resources, mapping hints.
- Build briefing** (build_context)
Single structured briefing (logs + metadata + operator error) for the model.
- LLM analysis** (llm_analysis)
OpenRouter JSON (or heuristic) — persisted in llm_audit.sqlite as llm_exchange rows.

LLM audit (llm_exchange)

Dashboard to monitor Agent lifecycle for Audit

LLM audit (llm_exchange)

- #2 · 2026-05-10T00:03:58.516195+00:00 · openrouter_primary
- model: deepseek/deepseek-chat · HTTP: 200
- ▶ Parsed response_json (model output)

▶ request_messages (system + user briefing)

▶ raw_assistant_text

Monitor runs (in-memory, this process)

```
[
  {
    "at": "2026-05-10T00:13:34.593967+00:00",
    "artifact_id": "Pattern_ContentBasedRouting_RaiseError",
    "message_guid": "AGn_ywqYTRdG7SQUxGFxo1ZBOfq",
    "skipped": true,
    "skip_reason": "duplicate_message_id_sqlite",
    "error": null
  },
  {
    "at": "2026-05-10T00:08:34.562336+00:00",
    "artifact_id": "Pattern_ContentBasedRouting_RaiseError",
    "message_guid": "AGn_ywqYTRdG7SQUxGFxo1ZBOfq",
    "skipped": true,
    "skip_reason": "duplicate_message_id_sqlite",
    "error": null
  },
]
```